

Wine

Homework #5

Naomi Buell

Richie Rivera

Alexander Simon

Nana Frimpong

Said Naqwe

2026-05-16

1 Introduction

In this homework assignment, we explore, analyze, and model a data set containing information on approximately 12,000 commercially available wines. Our objective is to build a count regression model to predict the number of cases of wine that will be sold given certain properties of the wine.

2 Data Exploration

```
# Load the training and evaluation datasets
wine_train <- read_csv("./data/wine-training-data.csv")
wine_eval <- read_csv("./data/wine-evaluation-data.csv")

# Count rows and columns
nrows <- wine_train |> nrow()
ncols <- wine_train |> length()

# Get missingness
perc_complete <- round(wine_train |> n_complete() / nrows / ncols * 100, 2)
```

The wine training data set contains 12795 rows and 16 features, and is 95.99 percent complete. We will need to address the missingness in the data preparation section. All variables are numeric; most are continuous variables describing chemical characteristics of the wines, while others like **STARS** and **LabelAppeal** are discrete. The target variable is **TARGET**, which represents the number of cases purchased.

Table 1: Summary statistics for all variables in the training dataset

Variable	Completeness						
	rate	Mean	Min	P25	P50	P75	Max
INDEX	1.00	8100.0000	1.00	4000.000	8.1e+03	1.2e+04	16000.0
TARGET	1.00	3.0000	0.00	2.000	3.0e+00	4.0e+00	8.0
FixedAcidity	1.00	7.1000	-18.00	5.200	6.9e+00	9.5e+00	34.0
VolatileAcidity	1.00	0.3200	-2.80	0.130	2.8e-01	6.4e-01	3.7
CitricAcid	1.00	0.3100	-3.20	0.030	3.1e-01	5.8e-01	3.9
ResidualSugar	0.95	5.4000	-130.00	-2.000	3.9e+00	1.6e+01	140.0
Chlorides	0.95	0.0550	-1.20	-0.031	4.6e-02	1.5e-01	1.4
FreeSulfurDiox- ide	0.95	31.0000	-560.00	0.000	3.0e+01	7.0e+01	620.0
TotalSulfurDiox- ide	0.95	120.0000	-820.00	27.000	1.2e+02	2.1e+02	1100.0
Density	1.00	0.9900	0.89	0.990	9.9e-01	1.0e+00	1.1
pH	0.97	3.2000	0.48	3.000	3.2e+00	3.5e+00	6.1
Sulphates	0.91	0.5300	-3.10	0.280	5.0e-01	8.6e-01	4.2
Alcohol	0.95	10.0000	-4.70	9.000	1.0e+01	1.2e+01	26.0
LabelAppeal	1.00	-0.0091	-2.00	-1.000	0.0e+00	1.0e+00	2.0
AcidIndex	1.00	7.8000	4.00	7.000	8.0e+00	8.0e+00	17.0
STARS	0.74	2.0000	1.00	1.000	2.0e+00	3.0e+00	4.0

Table 1 presents descriptive characteristics of the training data.

```
# Get correlation matrix
wine_train_cor <- wine_train |>
  dplyr::select(where(is.numeric)) |>
  cor(use = "pairwise.complete.obs")

wine_train_cor_plot <- wine_train_cor
wine_train_cor_plot[abs(wine_train_cor_plot) < 0.05] <- NA

corrplot(wine_train_cor_plot, method = "circle", type = "upper", sig.level = 0.05, insig = "bl")

most_corr_w_target_flag <- wine_train_cor |>
  as.data.frame() |>
  rownames_to_column(var = "Variable") |>
  filter(Variable != "TARGET") |>
  transmute(
```

```

Variable,
Correlation = TARGET,
Abs_Correlation = abs(TARGET)
) |>
arrange(desc(Abs_Correlation)) |>
filter(Abs_Correlation > 0.05) |>
pull(Variable)

```

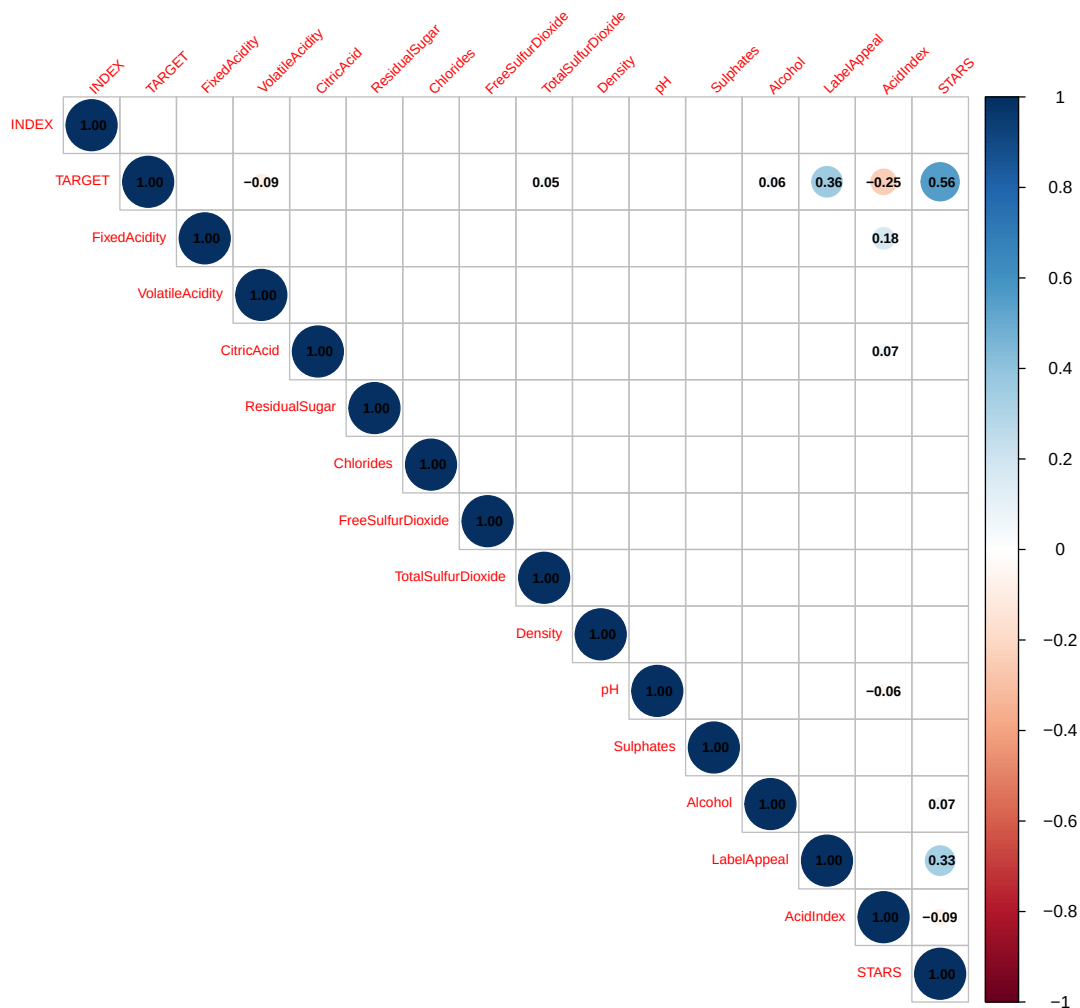


Figure 1: Correlation matrix of all variables in the training dataset. Note: Correlations with absolute value less than 0.05 are suppressed.

The correlation between all variables are shown Figure 1. The variables most correlated with **TARGET**, the number of cases purchased, are **STARS**, **LabelAppeal**, **AcidIndex**, **VolatileAcidity**, **Alcohol**, **TotalSulfurDioxide**. This means that wines with higher ratings (**STARS**), more appealing labels (**LabelAppeal**), lower acidity (**AcidIndex** and **VolatileAcidity**), higher alcohol content (**Alcohol**), and higher sulfur dioxide levels (**TotalSulfurDioxide**) tend to have higher sales. These variables will be important to consider in our modeling process, as they may have strong predictive power for the target variable.

It is important to note that label appeal and star ratings are likely to be highly correlated with each other, which may lead to multicollinearity issues in a regression model. Similarly, **AcidIndex** and **VolatileAcidity** are likely to be correlated, which may also lead to multicollinearity issues. We will need to consider this when selecting variables for our model.

Lastly, we also looked at the distribution of each variable to check for skewness and outliers.

```
# Keep only continuous
continuous_vars <- wine_train |>
  dplyr::select(where(is.numeric), -INDEX) |>
  names()

wine_train |>
  dplyr::select(all_of(continuous_vars)) |>
  pivot_longer(cols = everything(), names_to = "variable", values_to = "value") |>
  ggplot(aes(x = value)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +
  facet_wrap(~ variable, scales = "free", ncol = 4) +
  theme_minimal() +
  theme(panel.grid = element_blank()) +
  labs(x = NULL, y = "Count")
```

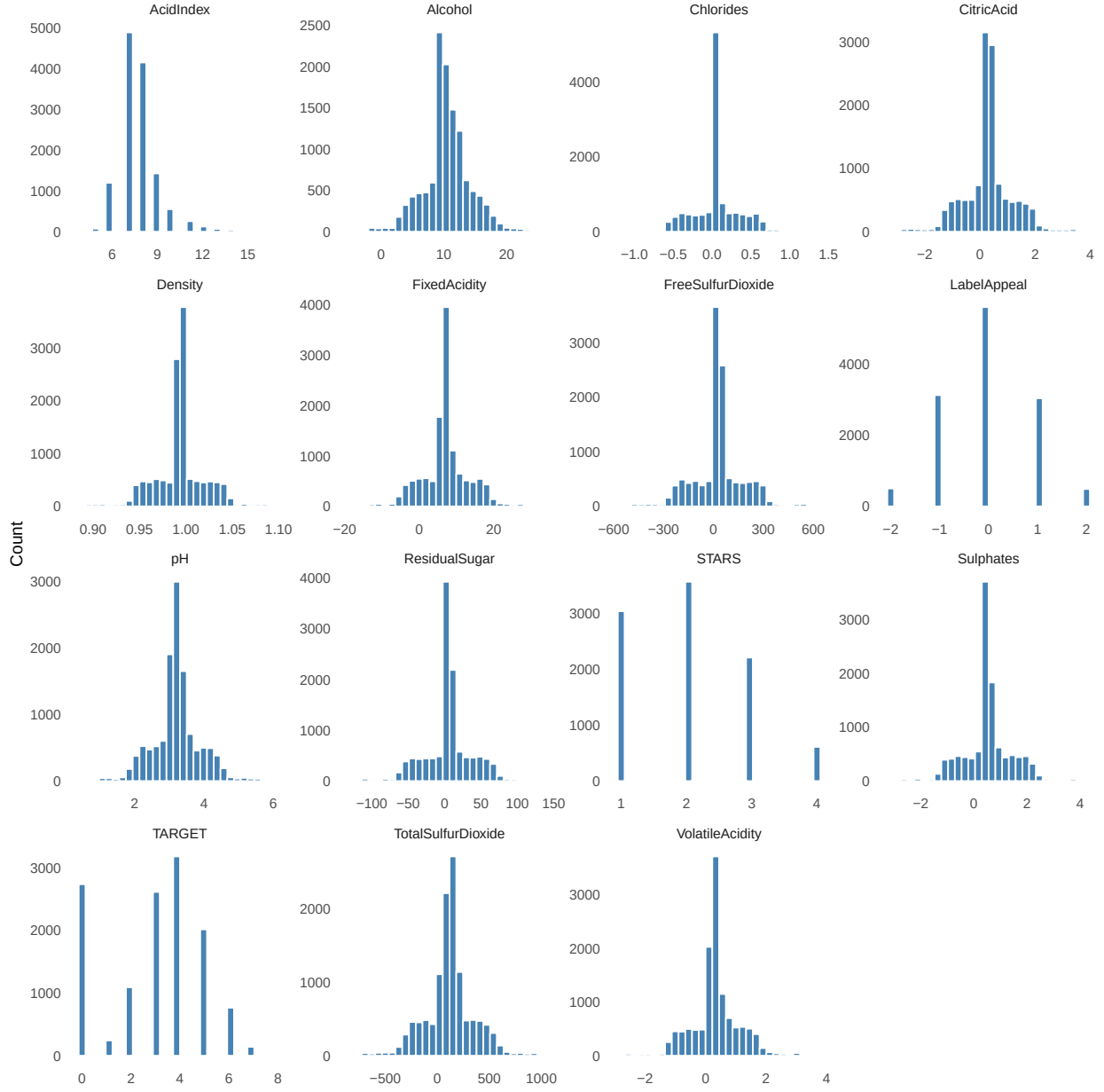


Figure 2: Distribution of continuous variables in the training dataset.

AcidIndex is right-skewed, with fewer lines having high acidity. Many other variables appear normally distributed, such as **Alcohol**, **pH**, and **TotalSulfurDioxide**. **STARS** is a discrete variable with values from 1 to 4, and is right-skewed with more wines having lower ratings. **LabelAppeal** is also discrete, with values from -2 to 2, and is normally distributed around 0. There do not appear to be any extreme outliers in the data, although many variables have high frequency of median values, which may indicate that many wines have similar characteristics in these dimensions. Note that **TARGET** has a high frequency of 0 values, which may indicate that many wines have no sales. This may require special consideration in our modeling process.

3 Data Preparation

For the wine data used in this assignment, the first step in data preparation was to examine whether any variables contained missing values. Unlike the crime dataset from Homework 3, the wine dataset contains missing values across several predictor variables in both the training and evaluation datasets. Because of this, appropriate missing-value treatment was required before proceeding to modeling.

To address missing data, median imputation was applied to all numeric predictor variables. Median imputation was selected because many of the chemical variables (such as sulfur dioxide levels, residual sugar, and alcohol content) are skewed and may contain outliers. Using the median provides a robust estimate that is less sensitive to extreme values than the mean.

In addition to imputation, missingness indicator variables were created for each predictor. This was done because the assignment specifically notes that missing values themselves may be predictive of the response variable. By creating binary flags indicating whether a value was originally missing, we preserve this potential signal for use in the modeling stage.

Next, several transformations were applied to improve the distribution of the predictors. Variables such as `alcohol`, `residualsugar`, `freesulfurdioxide`, and `totalsulfurdioxide` exhibited right-skewed distributions, so log transformations were created using `log1p()`. These transformations reduce skewness and help stabilize relationships between predictors and the count response variable.

We also created a set of engineered variables to capture relationships between predictors that may not be fully represented by the original variables. The variable `sulfur_ratio` measures the relationship between total and free sulfur dioxide, while `acidity_ratio` compares volatile acidity to fixed acidity. The variable `density_alcohol_ratio` captures the interaction between density and alcohol content. Finally, `quality_score` combines expert rating (`stars`) and marketing appeal (`labelappeal`) to reflect an overall perceived quality of the wine.

In addition, the `stars` variable was transformed into categorical buckets (low, medium, high) to capture potential non-linear effects. These buckets were then converted into dummy variables for use in regression models.

Finally, the cleaned training and evaluation datasets were stored as `wine_train_prep` and `wine_eval_prep` so they can be used directly in the modeling section.

```
# Remove INDEX variable and clean column names
wine_train_prep <- wine_train |>
  clean_names() |>
  dplyr::select(-any_of("index"))

wine_eval_prep <- wine_eval |>
  clean_names() |>
```

```

dplyr::select(-any_of("index"))

# Identify numeric predictors
numeric_vars <- wine_train_prep |>
  dplyr::select(where(is.numeric)) |>
  dplyr::select(-target) |>
  names()

# 1. Create missingness flags (MUST be done before we overwrite the NAs)
wine_train_prep <- wine_train_prep |>
  mutate(across(
    all_of(numeric_vars),
    ~ ifelse(is.na(.), 1, 0),
    .names = "{.col}_missing"
  ))

wine_eval_prep <- wine_eval_prep |>
  mutate(across(
    all_of(numeric_vars),
    ~ ifelse(is.na(.), 1, 0),
    .names = "{.col}_missing"
  ))

# 2. MEDIAN IMPUTATION (Moved up!)
# We impute the base variables first, so that when we engineer new features
# in the next step, there are no NAs to break the math.
for (v in numeric_vars) {
  med_val <- median(wine_train_prep[[v]], na.rm = TRUE)

  wine_train_prep[[v]][is.na(wine_train_prep[[v]])] <- med_val
  wine_eval_prep[[v]][is.na(wine_eval_prep[[v]])] <- med_val
}

# 3. Calculate the minimums from the TRAINING data (now that data is imputed)
min_alc <- min(wine_train_prep$alcohol, na.rm = TRUE)
min_sugar <- min(wine_train_prep$residual_sugar, na.rm = TRUE)
min_fsd <- min(wine_train_prep$free_sulfur_dioxide, na.rm = TRUE)
min_tsd <- min(wine_train_prep$total_sulfur_dioxide, na.rm = TRUE)

```

```
# 4. Transformations and feature engineering
wine_train_prep <- wine_train_prep |>
  mutate(
    # Shift values by subtracting the minimum before applying log1p
    log_alcohol = log1p(alcohol - min_alc),
    log_residual_sugar = log1p(residual_sugar - min_sugar),
    log_free_sulfur_dioxide = log1p(free_sulfur_dioxide - min_fsd),
    log_total_sulfur_dioxide = log1p(total_sulfur_dioxide - min_tsd),

    sulfur_ratio = total_sulfur_dioxide / ifelse(free_sulfur_dioxide + 1 == 0, 0.001, free_sulfur_dioxide + 1),
    acidity_ratio = volatile_acidity / ifelse(fixed_acidity + 1 == 0, 0.001, fixed_acidity + 1),
    density_alcohol_ratio = density / ifelse(alcohol + 1 == 0, 0.001, alcohol + 1),
    quality_score = stars * label_appeal,

    stars_bucket = case_when(
      stars <= 2 ~ "low",
      stars == 3 ~ "medium",
      stars >= 4 ~ "high",
      TRUE ~ "unknown"
    )
  )

wine_eval_prep <- wine_eval_prep |>
  mutate(
    # Apply the exact same training minimums to the evaluation set
    log_alcohol = log1p(alcohol - min_alc),
    log_residual_sugar = log1p(residual_sugar - min_sugar),
    log_free_sulfur_dioxide = log1p(free_sulfur_dioxide - min_fsd),
    log_total_sulfur_dioxide = log1p(total_sulfur_dioxide - min_tsd),

    sulfur_ratio = total_sulfur_dioxide / (free_sulfur_dioxide + 1),
    acidity_ratio = volatile_acidity / (fixed_acidity + 1),
    density_alcohol_ratio = density / (alcohol + 1),
    quality_score = stars * label_appeal,

    stars_bucket = case_when(
      stars <= 2 ~ "low",
      stars == 3 ~ "medium",
      stars >= 4 ~ "high",
    )
  )
```



```

    TRUE ~ "unknown"
  )
)

```

Warning in log1p(free_sulfur_dioxide - min_fsd): NaNs produced

```

# 5. Convert categorical bucket to dummy variables
wine_train_prep <- fastDummies::dummy_cols(
  wine_train_prep,
  select_columns = "stars_bucket",
  remove_selected_columns = TRUE,
  remove_first_dummy = TRUE
)

wine_eval_prep <- fastDummies::dummy_cols(
  wine_eval_prep,
  select_columns = "stars_bucket",
  remove_selected_columns = TRUE,
  remove_first_dummy = TRUE
)

# 6. Check for remaining missing values in training data
wine_train_prep |>
  summarise(across(everything(), ~ sum(is.na(.)))) |>
  pivot_longer(everything(), names_to = "Variable", values_to = "Missing") |>
  filter(Missing > 0)

```

```

# A tibble: 0 x 2
# i 2 variables: Variable <chr>, Missing <int>

```

```

# 7. Check for remaining missing values in evaluation data
wine_eval_prep |>
  summarise(across(everything(), ~ sum(is.na(.)))) |>
  pivot_longer(everything(), names_to = "Variable", values_to = "Missing") |>
  filter(Missing > 0)

```

```

# A tibble: 2 x 2
  Variable      Missing
  <chr>         <int>
1 target         3335
2 log_free_sulfur_dioxide 1

```

4 Assumptions for Count Regression Model

Count regression models rest on a set of assumptions that must be evaluated before selecting a final modeling approach. In this section, we assess the key assumptions relevant to Poisson regression, Negative Binomial regression, and their zero-inflated variants using the prepared training data.

4.1 Independence of Observations

Count regression models assume that the outcome for each observation is independent of all others. In the wine dataset, each row represents a distinct wine product, and there is no explicit grouping structure (e.g., repeated measures on the same wine or hierarchical nesting by producer) that would induce correlation between observations. We therefore treat the independence assumption as satisfied.

4.2 Distribution of the Target Variable

The target variable `TARGET` represents the number of wine cases purchased — a non-negative integer count. For Poisson regression, the response is assumed to follow a Poisson distribution, which requires the conditional mean to equal the conditional variance (equidispersion). For Negative Binomial regression, the variance is allowed to exceed the mean (overdispersion), making it more flexible for real-world count data.

```
dispersion_check <- wine_train_prep |>
  summarise(
    Mean      = mean(target, na.rm = TRUE),
    Variance   = var(target, na.rm = TRUE),
    Dispersion_Ratio = var(target, na.rm = TRUE) / mean(target, na.rm = TRUE)
  )

knitr::kable(dispersion_check, digits = 3)
```

Table 2: Mean, variance, and dispersion ratio of the `TARGET` variable

Mean	Variance	Dispersion_Ratio
3.029	3.711	1.225

A dispersion ratio substantially greater than 1 indicates overdispersion, which would violate the Poisson equidispersion assumption and favor a Negative Binomial specification instead. The dispersion ratio of 1.225 indicates mild overdispersion in the `TARGET` variable — the variance (3.711) exceeds the mean (3.029), which violates the Poisson assumption of equidispersion. While this is not extreme, it is sufficient to suggest that a Negative Binomial model may provide a

better fit than a standard Poisson model, as it accounts for this excess variation. We will evaluate both specifications in the modeling section and select the one that minimizes model deviance and information criteria.

4.3 Excess Zeros

Zero-inflated models (ZIP or ZINB) are appropriate when there is a higher frequency of zero counts than would be expected under a Poisson or Negative Binomial distribution alone. As noted in the Data Exploration section, **TARGET** exhibits a high concentration of zero values. We formally examine this by comparing the observed proportion of zeros against the theoretical proportion expected under a fitted Poisson model.

```
lambda_hat <- mean(wine_train_prep$target, na.rm = TRUE)

zero_check <- tibble(
  `Observed % Zeros` = round(mean(wine_train_prep$target == 0, na.rm = TRUE) * 100, 2),
  `Expected % Zeros (Poisson)` = round(exp(-lambda_hat) * 100, 2)
)

knitr::kable(zero_check)
```

Table 3: Observed vs. Poisson-expected proportion of zero counts in **TARGET**

Observed % Zeros	Expected % Zeros (Poisson)
21.37	4.84

If the observed proportion of zeros substantially exceeds the Poisson expectation, a zero-inflated model will be required to separately model the excess zero-generating process. The observed zero proportion (21.37%) far exceeds what a Poisson model would predict (4.84%), confirming significant excess zeros in **TARGET**. This suggests that a zero-inflated model such as ZIP or ZINB is likely needed to separately account for the portion of wines that generate no sales regardless of their chemical properties.

4.4 Log-Linearity

Count regression models with a log link assume a log-linear relationship between the predictors and the expected count. That is, the log of the expected **TARGET** should be approximately linear in each continuous predictor. We check this visually for the predictors most correlated with **TARGET** — **STARS**, **LabelAppeal**, **AcidIndex**, and **Alcohol** — by plotting the mean $\log(\text{TARGET} + 1)$ across binned predictor values.

```
key_vars <- c("stars", "label_appeal", "acid_index", "alcohol")

wine_train_prep |>
  mutate(log_target = log1p(target)) |>
  dplyr::select(log_target, all_of(key_vars)) |>
  pivot_longer(cols = all_of(key_vars), names_to = "variable", values_to = "value") |>
  mutate(bin = cut(value, breaks = 10)) |>
  group_by(variable, bin) |>
  summarise(mean_log_target = mean(log_target, na.rm = TRUE), .groups = "drop") |>
  ggplot(aes(x = bin, y = mean_log_target, group = variable)) +
  geom_line(color = "steelblue") +
  geom_point(color = "steelblue") +
  facet_wrap(~ variable, scales = "free_x", ncol = 2) +
  theme_minimal() +
  theme(
    axis.text.x = element_blank(),
    panel.grid = element_blank()
  ) +
  labs(x = "Binned Predictor Value", y = "Mean log(TARGET + 1)")
```

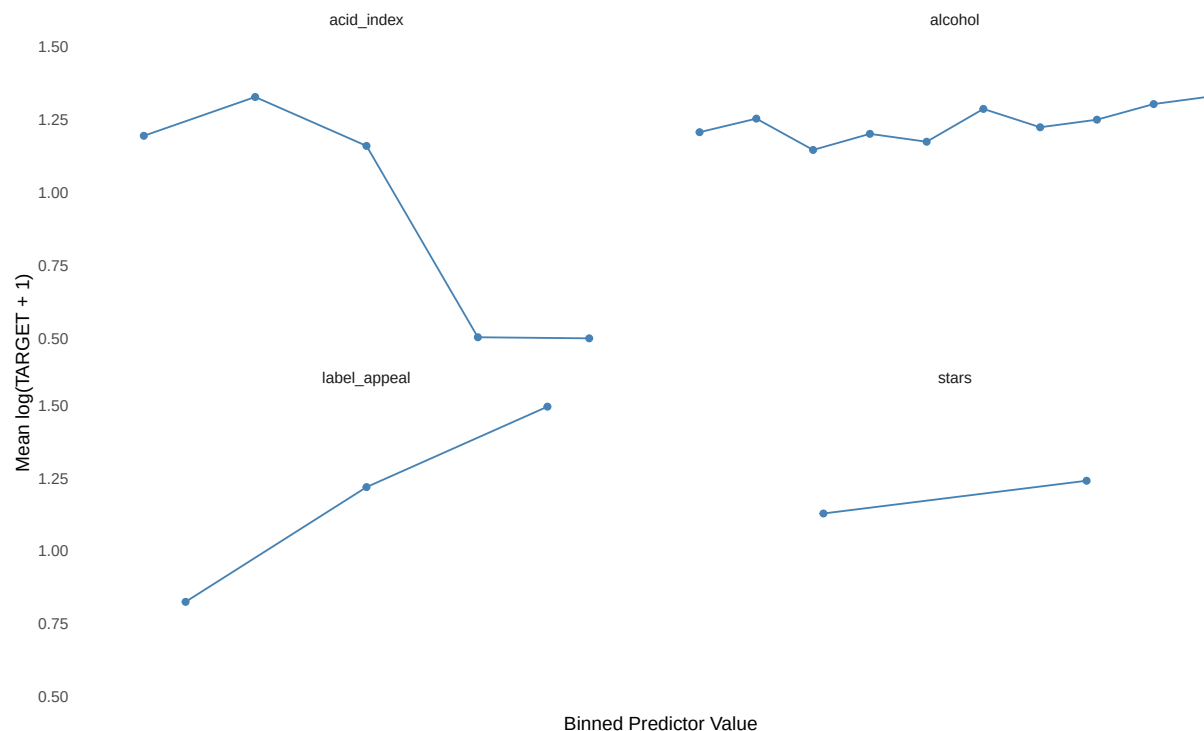


Figure 3: Mean log(TARGET + 1) across binned values of key predictors, used to assess the log-linearity assumption.

Approximate linearity in each panel supports the log-link assumption. Substantial curvature in any panel would suggest that a transformation or non-linear term for that predictor may be warranted. The log-linearity plots show mixed results across the four key predictors. `label_appeal` displays the clearest log-linear relationship, with mean $\log(\text{TARGET} + 1)$ rising steadily as label appeal increases, strongly supporting the log-link assumption for this variable. `alcohol` also shows an approximately linear trend with a gradual upward slope, though with minor fluctuations. `stars` shows a positive relationship but with only two visible data points, suggesting limited variation in this discrete variable. `acid_index`, however, shows a non-linear pattern — mean $\log(\text{TARGET} + 1)$ rises initially before dropping sharply at higher acidity levels — indicating that the log-linearity assumption may not hold for this variable and that a transformation or non-linear term for `acid_index` may improve model fit.

4.5 Multicollinearity

Count regression models, like all generalized linear models, are sensitive to high multicollinearity among predictors. As noted in the correlation matrix (Figure 1), `STARS` and `LabelAppeal` are both positively correlated with `TARGET`, and there is moderate correlation between the acidity variables. We compute Variance Inflation Factors (VIF) on a baseline Poisson model to formally assess multicollinearity.

```
baseline_formula <- target ~ stars + label_appeal + acid_index + alcohol +
  volatile_acidity + total_sulfur_dioxide +
  residual_sugar + chlorides + density + p_h

baseline_poisson <- glm(baseline_formula, data = wine_train_prep, family = poisson(link = "log"))

vif_vals <- vif(baseline_poisson) |>
  as.data.frame() |>
  rownames_to_column("Variable") |>
  rename(VIF = 2) |>
  arrange(desc(VIF))

knitr::kable(vif_vals, digits = 3)
```

Table 4: Variance Inflation Factors (VIF) for key predictors in a baseline Poisson model

Variable	VIF
stars	1.127
label_appeal	1.112
acid_index	1.019
alcohol	1.008

Table 4: Variance Inflation Factors (VIF) for key predictors in a baseline Poisson model

Variable	VIF
p_h	1.005
volatile_acidity	1.003
density	1.003
total_sulfur_dioxide	1.003
chlorides	1.002
residual_sugar	1.001

VIF values exceeding 5 (or conservatively 10) indicate problematic multicollinearity. Variables with high VIF may need to be excluded or combined in the final model specification. All VIF values are extremely close to 1, indicating virtually no multicollinearity among the predictors in the baseline Poisson model. No variable exceeds the conservative threshold of 5, confirming that each predictor contributes independent information to the model. This means multicollinearity will not be a concern during model building, and all variables can be retained without risk of inflated standard errors or unstable coefficient estimates.

4.6 Goodness of Fit

Finally, we assess the overall fit of the baseline Poisson model using the residual deviance relative to its degrees of freedom. A residual deviance substantially larger than the degrees of freedom is a further indicator of overdispersion or model misfit, and may motivate the use of a Negative Binomial or zero-inflated specification.

```
deviance_check <- tibble(
  `Residual Deviance` = round(deviance(baseline_poisson), 2),
  `Degrees of Freedom` = df.residual(baseline_poisson),
  `Deviance / df`      = round(deviance(baseline_poisson) / df.residual(baseline_poisson), 3)
)

knitr::kable(deviance_check)
```

Table 5: Residual deviance and degrees of freedom for the baseline Poisson model

Residual Deviance	Degrees of Freedom	Deviance / df
18452.11	12784	1.443

A deviance-to-df ratio near 1.0 is consistent with a well-fitting Poisson model. Ratios substantially above 1.0 confirm overdispersion and support transitioning to a Negative Binomial or zero-inflated

model in the modeling section. The deviance-to-df ratio of **1.443** confirms overdispersion in the baseline Poisson model — the residual deviance (18,452.11) substantially exceeds the degrees of freedom (12,784). This is consistent with our earlier finding that the variance exceeds the mean, and reinforces that a standard Poisson model is not the most appropriate fit for this data. A Negative Binomial or zero-inflated specification will be better suited to account for this excess variation in the modeling section.

5 Assumptions for Linear Regression

From the above, we have already addressed the Independence of Observations through the dataset structure and Multicollinearity using Variance Inflation Factors (VIF). To complete the standard linear regression diagnostic suite as outlined in our primary resources, we must evaluate constant variance, normality of errors, and the identification of influential observations.

1. Constant Variance (Homoscedasticity)

```
# Residuals vs Fitted values: Looking for a 'funnel' shape
plot(baseline_poisson, which = 1)

# Scale-Location plot: Uses square root of absolute residuals to better detect trends
plot(baseline_poisson, which = 3)
```

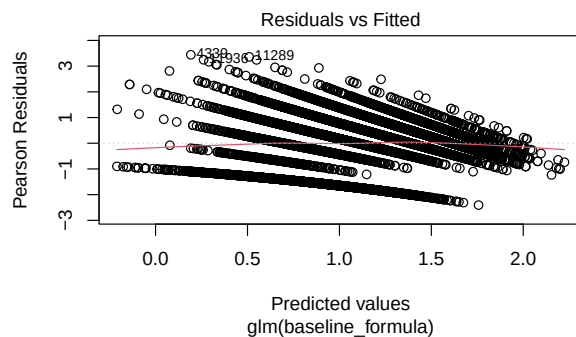


Figure 4: Standardized Residuals vs. Fitted values

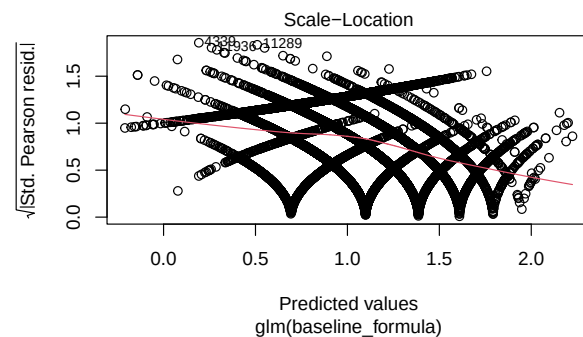


Figure 5: Standardized Residuals vs. Fitted values and the Scale-Location plot.

Based on the Standardized Residuals vs. Fitted values and the Scale-Location plot, the non-linear trend line and the “banded” vertical clusters indicate that the model is failing to capture the discrete nature of the count data, while the upward slope in the scale-location plot confirms heteroscedasticity, where the variance of the errors increases alongside the predicted number of wine cases.

2. Normality of Errors

```
# Normal Q-Q Plot: Points should follow the diagonal line[cite: 1, 2]
plot(baseline_poisson, which = 2)
```

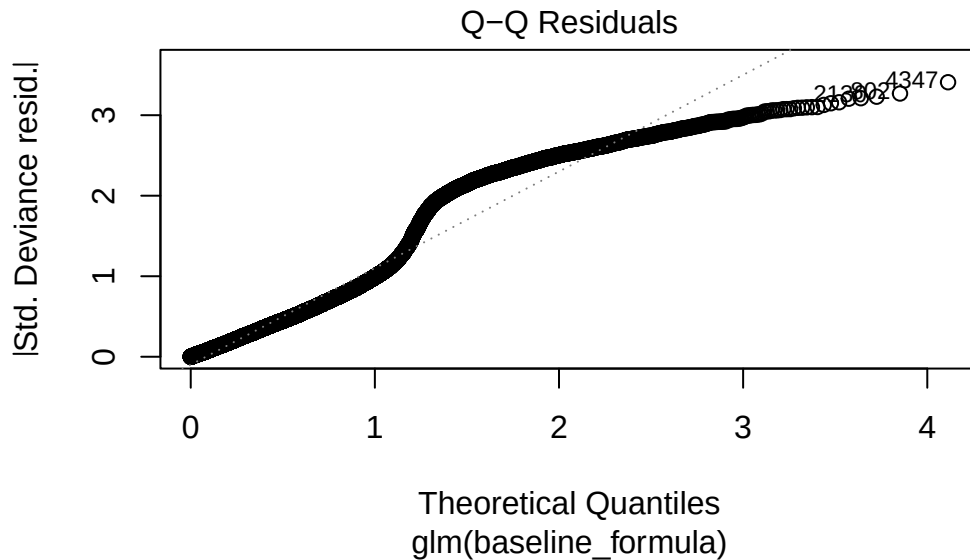


Figure 6: Normal Q-Q plot of the residuals.

Based on the QQ plot, the significant “S-shaped” deviations at both the lower and upper tails indicate that the residuals are not normally distributed, suggesting that the model is struggling to account for both the excess of zeros and the high-volume outliers in the dataset.

Because the fundamental assumptions of constant variance, linearity, and normality are clearly violated, proceeding with a standard linear regression is technically invalid for this data; however, we will proceed with the analysis while remaining cautious of biased p-values, unreliable confidence intervals, and the high likelihood of the model predicting impossible negative values for wine sales.

6 Build models

6.1 Modeling Plan

We follow a workflow centered on building valid models:

1. Partitioning: We split data into a training set for model development and a test set for assessing predictive ability.
2. Poisson GLM: Because our target variable is a count of cases purchased, we utilize Poisson regression. We will use the Pearson dispersion check to verify the model’s validity and check for overdispersion.

6.2 Data Splits

```
# Set a seed for reproducibility so you get the same split every time
set.seed(19940211)

# Calculate how many rows make up 80% of the dataset
train_size <- floor(0.80 * nrow(wine_train_prep))

# Randomly select row numbers for the training set
train_indices <- sample(seq_len(nrow(wine_train_prep)), size = train_size)

# Subset the dataset based on those row numbers
wine_train_split <- wine_train_prep[train_indices, ]
wine_test_split <- wine_train_prep[-train_indices, ]
```

6.3 Model 1. Full Poisson Model (All untransformed variables)

For our baseline model, we include all available predictor variables in their original, untransformed state. Because our **target** variable represents a count (number of cases purchased), we use a Poisson Generalized Linear Model (`family = "poisson"`).

This full model gives us a starting point to assess the baseline residual deviance, identify which predictors show initial statistical significance, and check for potential overdispersion before we evaluate simpler models.

```
model1 <- glm(
  formula = target ~ .,
  family = "poisson",
  data = wine_train_split
)

summary(model1)
```

Call:

```
glm(formula = target ~ ., family = "poisson", data = wine_train_split)
```

Coefficients: (6 not defined because of singularities)

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	1.253e+00	5.648e-01	2.218	0.02655 *
fixed_acidity	4.108e-04	9.157e-04	0.449	0.65367
volatile_acidity	-2.906e-02	7.303e-03	-3.979	6.91e-05 ***

citric_acid	6.059e-03	6.628e-03	0.914	0.36058
residual_sugar	1.752e-04	4.218e-04	0.415	0.67798
chlorides	-4.782e-02	1.843e-02	-2.595	0.00947 **
free_sulfur_dioxide	1.140e-04	9.191e-05	1.241	0.21473
total_sulfur_dioxide	8.223e-05	7.075e-05	1.162	0.24515
density	-2.446e-01	2.143e-01	-1.141	0.25385
p_h	-1.587e-02	8.561e-03	-1.854	0.06375 .
sulphates	-1.288e-02	6.414e-03	-2.009	0.04458 *
alcohol	6.710e-03	5.103e-03	1.315	0.18856
label_appeal	1.497e-01	1.904e-02	7.861	3.82e-15 ***
acid_index	-7.921e-02	5.115e-03	-15.486	< 2e-16 ***
stars	3.283e-01	1.620e-02	20.258	< 2e-16 ***
fixed_acidity_missing	NA	NA	NA	NA
volatile_acidity_missing	NA	NA	NA	NA
citric_acid_missing	NA	NA	NA	NA
residual_sugar_missing	1.783e-02	2.653e-02	0.672	0.50153
chlorides_missing	6.007e-03	2.599e-02	0.231	0.81723
free_sulfur_dioxide_missing	8.977e-03	2.622e-02	0.342	0.73206
total_sulfur_dioxide_missing	1.773e-02	2.555e-02	0.694	0.48777
density_missing	NA	NA	NA	NA
p_h_missing	-3.528e-02	3.407e-02	-1.035	0.30053
sulphates_missing	-9.864e-03	1.964e-02	-0.502	0.61554
alcohol_missing	2.985e-02	2.583e-02	1.156	0.24777
label_appeal_missing	NA	NA	NA	NA
acid_index_missing	NA	NA	NA	NA
stars_missing	-1.076e+00	2.031e-02	-52.951	< 2e-16 ***
log_alcohol	-5.104e-02	7.031e-02	-0.726	0.46785
log_residual_sugar	-1.408e-02	4.336e-02	-0.325	0.74536
log_free_sulfur_dioxide	-6.138e-03	4.095e-02	-0.150	0.88084
log_total_sulfur_dioxide	6.940e-03	5.499e-02	0.126	0.89958
sulfur_ratio	1.952e-07	6.304e-07	0.310	0.75687
acidity_ratio	1.278e-04	1.080e-04	1.183	0.23695
density_alcohol_ratio	1.092e-04	3.007e-04	0.363	0.71651
quality_score	4.163e-03	8.205e-03	0.507	0.61192
stars_bucket_low	4.271e-01	4.618e-02	9.249	< 2e-16 ***
stars_bucket_medium	2.192e-01	2.975e-02	7.369	1.72e-13 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

Null deviance: 18322 on 10235 degrees of freedom
 Residual deviance: 11028 on 10203 degrees of freedom
 AIC: 36605

Number of Fisher Scoring iterations: 6

6.4 Model 2. Simplified Poisson Model (Reduce Multicollinearity)

Our dataset contains several engineered variables that are direct mathematical combinations of base variables (e.g., `quality_score` is the product of `stars` and `label_appeal`; `sulfur_ratio` is derived from total and free sulfur dioxide). Including both the original base variables and their direct mathematical combinations can cause severe multicollinearity, which artificially inflates the standard errors of our coefficients.

In Model 2, we remove these highly redundant engineered interactions to stabilize our parameter estimates while keeping the primary effects intact.

```
# Fit Poisson model excluding highly collinear engineered features
model2 <- glm(
  formula = target ~ . - quality_score - sulfur_ratio - acidity_ratio - density_alcohol_ratio,
  family = "poisson",
  data = wine_train_split
)

summary(model2)
```

Call:

```
glm(formula = target ~ . - quality_score - sulfur_ratio - acidity_ratio -
    density_alcohol_ratio, family = "poisson", data = wine_train_split)
```

Coefficients: (6 not defined because of singularities)

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	1.274e+00	5.641e-01	2.259	0.0239 *
fixed_acidity	3.743e-04	9.143e-04	0.409	0.6823
volatile_acidity	-2.866e-02	7.291e-03	-3.931	8.45e-05 ***
citric_acid	6.078e-03	6.627e-03	0.917	0.3591
residual_sugar	1.781e-04	4.217e-04	0.422	0.6728
chlorides	-4.773e-02	1.843e-02	-2.590	0.0096 **
free_sulfur_dioxide	1.140e-04	9.190e-05	1.241	0.2147

total_sulfur_dioxide	8.197e-05	7.074e-05	1.159	0.2466
density	-2.468e-01	2.143e-01	-1.152	0.2494
p_h	-1.587e-02	8.560e-03	-1.854	0.0637 .
sulphates	-1.291e-02	6.413e-03	-2.013	0.0442 *
alcohol	7.085e-03	5.084e-03	1.394	0.1634
label_appeal	1.587e-01	6.889e-03	23.030	< 2e-16 ***
acid_index	-7.930e-02	5.114e-03	-15.508	< 2e-16 ***
stars	3.276e-01	1.608e-02	20.371	< 2e-16 ***
fixed_acidity_missing	NA	NA	NA	NA
volatile_acidity_missing	NA	NA	NA	NA
citric_acid_missing	NA	NA	NA	NA
residual_sugar_missing	1.752e-02	2.652e-02	0.661	0.5089
chlorides_missing	6.033e-03	2.598e-02	0.232	0.8164
free_sulfur_dioxide_missing	9.529e-03	2.621e-02	0.364	0.7162
total_sulfur_dioxide_missing	1.736e-02	2.555e-02	0.680	0.4968
density_missing	NA	NA	NA	NA
p_h_missing	-3.507e-02	3.404e-02	-1.030	0.3029
sulphates_missing	-1.018e-02	1.964e-02	-0.518	0.6043
alcohol_missing	3.114e-02	2.581e-02	1.207	0.2276
label_appeal_missing	NA	NA	NA	NA
acid_index_missing	NA	NA	NA	NA
stars_missing	-1.076e+00	2.031e-02	-52.958	< 2e-16 ***
log_alcohol	-5.674e-02	6.990e-02	-0.812	0.4169
log_residual_sugar	-1.399e-02	4.336e-02	-0.323	0.7470
log_free_sulfur_dioxide	-6.281e-03	4.094e-02	-0.153	0.8781
log_total_sulfur_dioxide	7.358e-03	5.499e-02	0.134	0.8936
stars_bucket_low	4.195e-01	4.306e-02	9.742	< 2e-16 ***
stars_bucket_medium	2.142e-01	2.745e-02	7.803	6.04e-15 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

Null deviance: 18322 on 10235 degrees of freedom
 Residual deviance: 11030 on 10207 degrees of freedom
 AIC: 36598

Number of Fisher Scoring iterations: 6

6.5 Model 3. Negative Binomial Model

In our Assumptions section, we calculated a dispersion ratio of 1.225, indicating that the variance of target slightly exceeds its mean. While a standard Poisson model assumes the variance equals the mean, a Negative Binomial model includes an extra parameter to explicitly handle this overdispersion.

For Model 3, we use the `glm.nb()` function from the MASS package. We will apply the same reduced feature set used in Model 2 to ensure the model remains stable and free from severe multicollinearity while we test this new distribution.

```
# Fit a Negative Binomial model to account for mild overdispersion
model3 <- MASS::glm.nb(
  formula = target ~ . - quality_score - sulfur_ratio - acidity_ratio - density_alcohol_ratio
  data = wine_train_split
)
```

```
Warning in theta.ml(Y, mu, sum(w), w, limit = control$maxit, trace =
control$trace > : iteration limit reached
```

```
Warning in theta.ml(Y, mu, sum(w), w, limit = control$maxit, trace =
control$trace > : iteration limit reached
```

```
summary(model3)
```

Call:

```
MASS::glm.nb(formula = target ~ . - quality_score - sulfur_ratio -
  acidity_ratio - density_alcohol_ratio, data = wine_train_split,
  init.theta = 40919.34745, link = log)
```

Coefficients: (6 not defined because of singularities)

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	1.274e+00	5.641e-01	2.259	0.0239 *
fixed_acidity	3.743e-04	9.144e-04	0.409	0.6823
volatile_acidity	-2.866e-02	7.291e-03	-3.931	8.45e-05 ***
citric_acid	6.078e-03	6.627e-03	0.917	0.3591
residual_sugar	1.781e-04	4.217e-04	0.422	0.6729
chlorides	-4.773e-02	1.843e-02	-2.590	0.0096 **
free_sulfur_dioxide	1.140e-04	9.191e-05	1.241	0.2147
total_sulfur_dioxide	8.197e-05	7.074e-05	1.159	0.2466
density	-2.468e-01	2.143e-01	-1.152	0.2495
p_h	-1.587e-02	8.560e-03	-1.854	0.0637 .
sulphates	-1.291e-02	6.414e-03	-2.013	0.0442 *
alcohol	7.085e-03	5.084e-03	1.394	0.1634

label_appeal	1.587e-01	6.889e-03	23.029	< 2e-16 ***
acid_index	-7.931e-02	5.114e-03	-15.508	< 2e-16 ***
stars	3.276e-01	1.608e-02	20.370	< 2e-16 ***
fixed_acidity_missing	NA	NA	NA	NA
volatile_acidity_missing	NA	NA	NA	NA
citric_acid_missing	NA	NA	NA	NA
residual_sugar_missing	1.752e-02	2.652e-02	0.661	0.5089
chlorides_missing	6.032e-03	2.598e-02	0.232	0.8164
free_sulfur_dioxide_missing	9.530e-03	2.621e-02	0.364	0.7162
total_sulfur_dioxide_missing	1.737e-02	2.555e-02	0.680	0.4968
density_missing	NA	NA	NA	NA
p_h_missing	-3.508e-02	3.405e-02	-1.030	0.3029
sulphates_missing	-1.018e-02	1.964e-02	-0.518	0.6043
alcohol_missing	3.114e-02	2.581e-02	1.207	0.2276
label_appeal_missing	NA	NA	NA	NA
acid_index_missing	NA	NA	NA	NA
stars_missing	-1.076e+00	2.031e-02	-52.957	< 2e-16 ***
log_alcohol	-5.674e-02	6.990e-02	-0.812	0.4170
log_residual_sugar	-1.399e-02	4.337e-02	-0.323	0.7470
log_free_sulfur_dioxide	-6.283e-03	4.094e-02	-0.153	0.8780
log_total_sulfur_dioxide	7.358e-03	5.500e-02	0.134	0.8936
stars_bucket_low	4.195e-01	4.306e-02	9.741	< 2e-16 ***
stars_bucket_medium	2.142e-01	2.745e-02	7.803	6.06e-15 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Negative Binomial(40919.35) family taken to be 1)

Null deviance: 18321 on 10235 degrees of freedom
 Residual deviance: 11030 on 10207 degrees of freedom
 AIC: 36601

Number of Fisher Scoring iterations: 1

Theta: 40919

Std. Err.: 38924

Warning while fitting theta: iteration limit reached

2 x log-likelihood: -36540.74

6.6 Model 4. Negative Binomial Model With Backward Stepwise Selection

To balance model complexity and goodness-of-fit, we perform backward stepwise selection starting from our Negative Binomial model (`model3`). This algorithm evaluates the model by iteratively removing the least significant predictors based on the Akaike Information Criterion (AIC). It continues this process until it finds the optimal sub-model where removing any more variables would hurt the model's predictive power more than it would help its simplicity.

```
# Perform backward stepwise selection
# (trace = 0 hides the massive console output during the iterative process)
model4 <- step(model3, direction = "backward", trace = 0)
```

```
Warning in theta.ml(Y, mu, sum(w), w, limit = control$maxit, trace =
control$trace > : iteration limit reached
Warning in theta.ml(Y, mu, sum(w), w, limit = control$maxit, trace =
control$trace > : iteration limit reached
```

```
summary(model4)
```

Call:

```
MASS::glm.nb(formula = target ~ fixed_acidity + volatile_acidity +
  citric_acid + residual_sugar + chlorides + free_sulfur_dioxide +
  total_sulfur_dioxide + density + p_h + sulphates + alcohol +
  label_appeal + acid_index + stars + fixed_acidity_missing +
  volatile_acidity_missing + citric_acid_missing + residual_sugar_missing +
  chlorides_missing + free_sulfur_dioxide_missing + total_sulfur_dioxide_missing +
  density_missing + p_h_missing + sulphates_missing + alcohol_missing +
  label_appeal_missing + stars_missing + log_alcohol + log_residual_sugar +
  log_free_sulfur_dioxide + log_total_sulfur_dioxide + stars_bucket_low +
  stars_bucket_medium, data = wine_train_split, init.theta = 40919.28074,
  link = log)
```

Coefficients: (5 not defined because of singularities)

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	1.274e+00	5.641e-01	2.259	0.0239 *
fixed_acidity	3.743e-04	9.144e-04	0.409	0.6823
volatile_acidity	-2.866e-02	7.291e-03	-3.931	8.45e-05 ***
citric_acid	6.078e-03	6.627e-03	0.917	0.3591
residual_sugar	1.781e-04	4.217e-04	0.422	0.6729
chlorides	-4.773e-02	1.843e-02	-2.590	0.0096 **
free_sulfur_dioxide	1.140e-04	9.191e-05	1.241	0.2147

total_sulfur_dioxide	8.197e-05	7.074e-05	1.159	0.2466
density	-2.468e-01	2.143e-01	-1.152	0.2495
p_h	-1.587e-02	8.560e-03	-1.854	0.0637 .
sulphates	-1.291e-02	6.414e-03	-2.013	0.0442 *
alcohol	7.085e-03	5.084e-03	1.394	0.1634
label_appeal	1.587e-01	6.889e-03	23.029	< 2e-16 ***
acid_index	-7.931e-02	5.114e-03	-15.508	< 2e-16 ***
stars	3.276e-01	1.608e-02	20.370	< 2e-16 ***
fixed_acidity_missing	NA	NA	NA	NA
volatile_acidity_missing	NA	NA	NA	NA
citric_acid_missing	NA	NA	NA	NA
residual_sugar_missing	1.752e-02	2.652e-02	0.661	0.5089
chlorides_missing	6.032e-03	2.598e-02	0.232	0.8164
free_sulfur_dioxide_missing	9.530e-03	2.621e-02	0.364	0.7162
total_sulfur_dioxide_missing	1.737e-02	2.555e-02	0.680	0.4968
density_missing	NA	NA	NA	NA
p_h_missing	-3.508e-02	3.405e-02	-1.030	0.3029
sulphates_missing	-1.018e-02	1.964e-02	-0.518	0.6043
alcohol_missing	3.114e-02	2.581e-02	1.207	0.2276
label_appeal_missing	NA	NA	NA	NA
stars_missing	-1.076e+00	2.031e-02	-52.957	< 2e-16 ***
log_alcohol	-5.674e-02	6.990e-02	-0.812	0.4170
log_residual_sugar	-1.399e-02	4.337e-02	-0.323	0.7470
log_free_sulfur_dioxide	-6.283e-03	4.094e-02	-0.153	0.8780
log_total_sulfur_dioxide	7.358e-03	5.500e-02	0.134	0.8936
stars_bucket_low	4.195e-01	4.306e-02	9.741	< 2e-16 ***
stars_bucket_medium	2.142e-01	2.745e-02	7.803	6.06e-15 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Negative Binomial(40919.28) family taken to be 1)

Null deviance: 18321 on 10235 degrees of freedom
 Residual deviance: 11030 on 10207 degrees of freedom
 AIC: 36601

Number of Fisher Scoring iterations: 1

Theta: 40919

Std. Err.: 38924

Warning while fitting theta: iteration limit reached

2 x log-likelihood: -36540.74

6.7 Model 5. Full Multiple Linear Regression Model

Our baseline multiple linear regression model used all un-transformed predictors.

```
model5 <- lm(
  formula = target ~ .,
  data = wine_train_split
)

summary(model5)
```

Call:

```
lm(formula = target ~ ., data = wine_train_split)
```

Residuals:

Min	1Q	Median	3Q	Max
-4.5907	-0.8756	0.0101	0.8606	5.7065

Coefficients: (6 not defined because of singularities)

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	3.025e+00	1.228e+00	2.463	0.013794	*
fixed_acidity	1.737e-03	2.081e-03	0.835	0.403796	
volatile_acidity	-8.774e-02	1.655e-02	-5.302	1.17e-07	***
citric_acid	1.948e-02	1.511e-02	1.289	0.197326	
residual_sugar	1.095e-04	9.382e-04	0.117	0.907049	
chlorides	-1.504e-01	4.173e-02	-3.605	0.000313	***
free_sulfur_dioxide	3.188e-04	2.164e-04	1.473	0.140815	
total_sulfur_dioxide	2.613e-04	1.468e-04	1.780	0.075168	.
density	-6.984e-01	4.873e-01	-1.433	0.151792	
p_h	-4.485e-02	1.940e-02	-2.312	0.020792	*
sulphates	-3.510e-02	1.457e-02	-2.408	0.016049	*
alcohol	2.189e-02	1.171e-02	1.869	0.061703	.
label_appeal	1.439e-01	4.392e-02	3.275	0.001059	**
acid_index	-1.966e-01	1.022e-02	-19.236	< 2e-16	***
stars	1.087e+00	3.698e-02	29.390	< 2e-16	***

```

fixed_acidity_missing      NA      NA      NA      NA
volatile_acidity_missing   NA      NA      NA      NA
citric_acid_missing        NA      NA      NA      NA
residual_sugar_missing     5.574e-02 6.086e-02 0.916 0.359796
chlorides_missing          1.645e-02 5.914e-02 0.278 0.780930
free_sulfur_dioxide_missing 2.755e-02 5.914e-02 0.466 0.641288
total_sulfur_dioxide_missing 5.157e-02 5.799e-02 0.889 0.373901
density_missing            NA      NA      NA      NA
p_h_missing                -8.313e-02 7.586e-02 -1.096 0.273130
sulphates_missing          -2.521e-02 4.435e-02 -0.568 0.569753
alcohol_missing            9.788e-02 5.915e-02 1.655 0.097971 .
label_appeal_missing       NA      NA      NA      NA
acid_index_missing         NA      NA      NA      NA
stars_missing              -2.370e+00 3.586e-02 -66.091 < 2e-16 ***
log_alcohol                -1.579e-01 1.602e-01 -0.986 0.324222
log_residual_sugar         8.980e-03 9.533e-02 0.094 0.924954
log_free_sulfur_dioxide    -3.472e-03 9.682e-02 -0.036 0.971394
log_total_sulfur_dioxide   -8.610e-03 1.102e-01 -0.078 0.937706
sulfur_ratio               6.081e-07 1.427e-06 0.426 0.669965
acidity_ratio              4.556e-04 2.886e-04 1.579 0.114401
density_alcohol_ratio      4.120e-04 7.877e-04 0.523 0.600967
quality_score              1.600e-01 2.070e-02 7.729 1.19e-14 ***
stars_bucket_low           1.132e+00 1.166e-01 9.711 < 2e-16 ***
stars_bucket_medium        5.714e-01 8.168e-02 6.995 2.82e-12 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Residual standard error: 1.309 on 10203 degrees of freedom

Multiple R-squared: 0.5394, Adjusted R-squared: 0.5379

F-statistic: 373.4 on 32 and 10203 DF, p-value: < 2.2e-16

6.8 Model 6. Multiple Linear Regression Model With Backward Stepwise Selection

For this model, we performed backward stepwise selection from the full multiple linear regression model to identify the most important predictors.

```

model6 <- step(model5, direction = "backward", trace = 0)

summary(model6)

```

Call:

```
lm(formula = target ~ volatile_acidity + chlorides + free_sulfur_dioxide +
    total_sulfur_dioxide + density + p_h + sulphates + alcohol +
    label_appeal + acid_index + stars + alcohol_missing + stars_missing +
    acidity_ratio + quality_score + stars_bucket_low + stars_bucket_medium,
    data = wine_train_split)
```

Residuals:

Min	1Q	Median	3Q	Max
-4.6145	-0.8733	0.0117	0.8643	5.7065

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	2.693e+00	5.204e-01	5.174	2.33e-07	***
volatile_acidity	-8.792e-02	1.653e-02	-5.319	1.07e-07	***
chlorides	-1.511e-01	4.169e-02	-3.624	0.000292	***
free_sulfur_dioxide	3.148e-04	8.967e-05	3.511	0.000449	***
total_sulfur_dioxide	2.527e-04	5.765e-05	4.384	1.18e-05	***
density	-7.192e-01	4.865e-01	-1.478	0.139316	
p_h	-4.416e-02	1.937e-02	-2.279	0.022667	*
sulphates	-3.475e-02	1.456e-02	-2.387	0.017005	*
alcohol	1.082e-02	3.552e-03	3.046	0.002323	**
label_appeal	1.431e-01	4.387e-02	3.262	0.001112	**
acid_index	-1.943e-01	1.003e-02	-19.370	< 2e-16	***
stars	1.087e+00	3.693e-02	29.432	< 2e-16	***
alcohol_missing	9.245e-02	5.885e-02	1.571	0.116243	
stars_missing	-2.372e+00	3.580e-02	-66.257	< 2e-16	***
acidity_ratio	4.642e-04	2.876e-04	1.614	0.106585	
quality_score	1.602e-01	2.068e-02	7.748	1.02e-14	***
stars_bucket_low	1.132e+00	1.164e-01	9.718	< 2e-16	***
stars_bucket_medium	5.702e-01	8.161e-02	6.987	2.98e-12	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.309 on 10218 degrees of freedom

Multiple R-squared: 0.539, Adjusted R-squared: 0.5383

F-statistic: 702.9 on 17 and 10218 DF, p-value: < 2.2e-16

7 Select Models

We evaluated the performance of the six candidate models using three regression metrics (RMSE, MSE, and MAE) and two goodness-of-fit criteria (AIC and BIC). These metrics were selected to enable comparison of both GLM and multiple linear regression models.

```
# Create custom helper functions to calculate model performance metrics
get_performance_metrics <- function(model, model_name, data, target_var = "target") {
  preds <- predict(model, newdata = data)
  actuals <- data[[target_var]]

  # Calculate metrics
  mse <- mean((actuals - preds)^2)
  rmse <- sqrt(mse)
  mae <- mean(abs(actuals - preds))

  tibble(
    Model = model_name,
    MSE = mse,
    RMSE = rmse,
    MAE = mae,
    AIC = AIC(model),
    BIC = BIC(model)
  )
}

# Evaluate models
models_list <- list(
  "Full Poisson Model" = model1,
  "Simplified Poisson Model" = model2,
  "Negative Binomial Model" = model3,
  "Negative Binomial Model with Backward Selection" = model4,
  "Full Multiple Linear Regression Model" = model5,
  "Stepwise Multiple Linear Regression Model" = model6
)

# Apply the performance evaluation function using test data
model_comparison <- imap_dfr(models_list, ~ {
  get_performance_metrics(.x, .y, wine_test_split)
}) |> arrange(desc(RMSE))
```

```
# Transpose the data and convert to a data frame
model_t <- as.data.frame(t(model_comparison))

# Display the results dataframe
knitr::kable(
  model_t,
  digits = 4,
  caption = "Performance of Candidate Models on Test Dataset"
)
```

Overall, the multiple linear regression models performed better than the count regression models. In particular, the stepwise multiple linear regression model had the lowest AIC and BIC scores, indicating that it is the most parsimonious model. Although its RMSE, MSE, and MAE were marginally higher than those of the full multiple linear regression model, the stepwise linear model is preferred because it is a simpler model.

However, because we need to deploy a count regression model (per assignment instructions), we select the negative binomial model as the final model. Since all of the candidate count regression models had similar performance metrics, we made this decision based on the known robustness of the negative binomial to overdispersion, which we observed during data exploration.

```
# Display the summary of the negative binomial model to interpret coefficients and significance
summary(model3)
```

To make inferences from the negative binomial model, we can look at the significance of the predictors. Significant predictors of the number of wine cases purchased include wine acidity (volatile_acidity and acid_index), label_appeal, and expert ratings (stars, stars_missing, stars_bucket_low, and stars_bucket_medium), which were all significant at or below the 0.001 level. In addition, chlorides and sulphates were significant at the 0.01 and 0.05 level, respectively. These predictors are consistent with scientific knowledge and business experience, since acidity, chlorides, and sulphates are known to influence wine flavors,¹ and product labels and ratings are known to influence the sales of many products. Ultimately, people tend to buy more of what they like and what others recommend.

8 Results

```
# Generate predictions on the evaluation dataset using the selected negative binomial model
eval_predictions <- predict(model3, newdata = wine_eval_prep, type = "response")
```

¹Crook C. (2024). Fine-tune your wine. Perfecting water composition. <https://www.murphyandson.co.uk/fine-tune-your-wine-perfecting-water-composition/>

```
# Create results dataframe
wine_eval_results <- wine_eval_prep |>
  dplyr::select(-c(target)) |>
  mutate(
    PREDICTED_TARGET = floor(eval_predictions)
  )

# Write predictions to csv file
write_csv(wine_eval_results, "wine_evaluation_predictions.csv")

# Display the first 10 predictions with key significant predictors from the negative binomial model
wine_eval_results |>
  dplyr::select(volatile_acidity, acid_index, chlorides, sulphates,
                label_appeal, stars, PREDICTED_TARGET) |>
  head(10) |>
  knitr::kable(
    caption = "Excerpts of Negative Binomial Model Predictions for Evaluation Set"
  )
```

Given our evaluation set and our negative binomial model, we are able to predict the number of wine cases that will be purchased. The first 10 predictions are shown in the table above (see CSV file included with assignment submission for all predictions). The predicted number of cases purchased (PREDICTED_TARGET) is consistent with expectations. For example, we would expect that fewer cases of wines with relatively low label appeal or expert ratings (stars) would be sold than those with high label appeal or expert ratings.

9 Conclusion

In conclusion, we selected the negative binomial model as the optimal count regression model for the wine data based on a combination of model performance and robustness to overdispersion effects, which we observed in the data. The significant predictors in this model suggest that the number of cases of wine purchased (target variable) is driven by several chemical properties of wine that likely influence its taste as well as the appeal of the product label and expert ratings. These inferences are also consistent with those from the stepwise multiple linear regression model, which had the best overall performance even though it was not a deployable model. The consistency among significant predictors from multiple models strengthens the hypothesis that these are important factors of wine sales. These findings could help wine stores and restaurants optimize sales, orders, and inventory for different wines.